

Simulated Annealing

Explicação do código, bloco por bloco

desafio Natura · `src/solver/heuristics/simulated_annealing.py`
Implementação da Seção 4 da formulação matemática.

Este documento tem duas partes. A **Parte 1** explica o que é a metaheurística Simulated Annealing (o conceito, sem código). A **Parte 2** percorre o arquivo inteiro, função por função, ligando cada trecho à seção correspondente do PDF da formulação.

Parte 1 — O que é Simulated Annealing

1.1 A analogia física (de onde vem o nome)

Annealing (recozimento) é um processo da metalurgia: aquece-se um metal até quase derreter e depois resfria-se **bem devagar**. Quente, os átomos se movem muito e ficam desorganizados; conforme esfria devagar, eles se acomodam numa estrutura cristalina de **baixa energia** (menos defeitos, mais resistente). Resfriar rápido demais congela os átomos numa bagunça — estrutura de energia alta, quebradiça.

O Simulated Annealing copia isso para otimização:

Metalurgia	Otimização
estado dos átomos	uma solução X (qual combinação cada setor usa)
energia do material	$E(X)$ — o quão “ruim” a solução é
temperatura	um número T que controla o quanto aceitamos piorar
resfriar devagar	reduzir T ao longo das iterações
estrutura final de baixa energia	a melhor solução encontrada

1.2 O problema que o SA resolve

Métodos exatos (o MIP no Gurobi) **garantem** o ótimo, mas podem demorar demais em problemas NP-difíceis grandes — o PDF classifica este como GAP (Generalized Assignment Problem), NP-difícil. O SA **abre mão da garantia** de otimalidade em troca de velocidade: acha uma solução boa (não necessariamente a melhor) em tempo controlado.

1.3 Por que não só “descer a ladeira” (hill-climbing)

A ideia mais simples seria sempre aceitar movimentos que melhoram e nunca os que pioram. Isso é *hill-climbing*. O problema: você fica **preso no primeiro vale** que encontrar (mínimo local), mesmo que exista um vale muito mais fundo do outro lado de uma “montanha”.

O SA escapa disso **aceitando pioras de propósito**, principalmente no começo:

- **Temperatura alta (início):** aceita quase qualquer piora — explora o espaço todo, pula montanhas.
- **Temperatura média:** aceita pioras pequenas, rejeita as grandes — vai afunilando.
- **Temperatura baixa (fim):** só aceita melhora — vira hill-climbing puro, “assenta” no fundo do vale.

1.4 A regra de Metropolis (o coração do SA)

É a fórmula que decide se aceita um movimento que piora:

- Se o movimento **melhora** ($\Delta E < 0$): aceita sempre.
- Se o movimento **piora** ($\Delta E > 0$): aceita com probabilidade $e^{(-\Delta E/T)}$.

Comportamento da exponencial: T grande $\rightarrow$ expoente perto de 0 $\rightarrow e^0 = 1 \rightarrow$ aceita quase sempre. T pequeno $\rightarrow$ expoente muito negativo $\rightarrow e^{(\text{muito negativo})} \approx 0 \rightarrow$ quase nunca aceita. Piora grande (ΔE grande) $\rightarrow$ menos provável de aceitar que piora pequena. É isso que dá ao SA a capacidade de escapar de mínimos locais no começo e convergir no fim.

Parte 2 — O código, linha por linha

2.1 Arquitetura do arquivo

O arquivo tem três camadas:

Camada	Funções	Papel
API pública	<code>solve()</code> , <code>AnnealingParams</code> , <code>AnnealingResult</code>	o que quem usa o módulo chama
Núcleo do SA	<code>_anneal</code> , <code>_step</code> , <code>_accepts</code> , <code>_stagnated</code> , <code>_neighbor</code> , <code>_evaluate</code>	algoritmo em si
Construção	<code>_build_problem</code> + <code>_demand_array</code> , <code>_capacity_array</code> , <code>_transform</code>	transforma os dicionários de entrada em arrays numpy

Separar em funções pequenas nomeadas é a convenção do repo (function-design.md): cada função faz uma coisa só, e o nome dela substitui o comentário.

2.2 Cabeçalho e imports (linhas 1-13)

```
"""Simulated Annealing pra atribuicao setor -> Bloco/Subbloco (Secao 4 do desafio Natura)."""
```

```
from __future__ import annotations
```

```
import math
import random
from collections import deque
from collections.abc import Iterable, Mapping, Sequence
from dataclasses import dataclass, field
```

```
import numpy as np
```

- `from __future__ import annotations` — faz o Python tratar as anotações de tipo como texto, sem avaliar na hora. Permite escrever `AnnealingResult` numa função antes da classe existir. É o padrão do repo.
- `math` — para o e^x (`math.exp`) na regra de Metropolis.
- `random` — gerador de números aleatórios (`random.Random`).
- `deque` — a “janela deslizando” de energias.
- `Iterable`, `Mapping`, `Sequence` — tipos genéricos das anotações.
- `dataclass`, `field` — criam classes de dados sem escrever o construtor.
- `numpy as np` — todos os cálculos vetorizados.

ProjectedDemand — apelido de tipo (linha 13)

```
ProjectedDemand = Mapping[tuple[int, int, int], float] # (setor, dia, combinacao) -> qtd
```

Um **apelido de tipo**: `ProjectedDemand` = “um dicionário cuja chave é o trio (setor, dia, combinação) e o valor é a demanda”. É só para não repetir `Mapping[tuple[int, int, int], float]` em todo lugar. É o `q(s,a,d)` do PDF. “Combinação” aqui = um par (Bloco, Subbloco), o `d` do conjunto `D` da Seção 3.1.

2.3 AnnealingParams (linhas 16-28)

```
@dataclass(frozen=True)
class AnnealingParams:
    initial_temperature: float = 1000.0 # T0
    cooling_rate: float = 0.99 # alfa
    min_temperature: float = 0.01 # T_min
    max_iterations: int = 100_000 # MaxIter
    stagnation_window: int = 200 # L
    stagnation_tolerance: float = 1e-5 # epsilon
    sector_bias: float = 0.8 # beta
    destination_bias: float = 0.7 # gama
    penalty_coefficient: float = 1e6 # rho
```

O que é `@dataclass`: um **decorador** que gera automaticamente o construtor (`__init__`), a representação para print (`__repr__`) e a comparação com `==` (`__eq__`). Você declara os campos, ele escreve a mecânica. `frozen=True` torna os campos imutáveis depois de criados.

Cada campo é um parâmetro da Seção 4.1 do PDF, com valor padrão:

Campo	Símbolo	O que faz
<code>initial_temperature = 1000.0</code>	T0	temperatura de partida
<code>cooling_rate = 0.99</code>	alfa	fator que multiplica T a cada iteração
<code>min_temperature = 0.01</code>	T_min	abaixo disso, para
<code>max_iterations = 100_000</code>	MaxIter	teto absoluto de iterações
<code>stagnation_window = 200</code>	L	tamanho da janela de detecção de estagnação
<code>stagnation_tolerance = 1e-5</code>	epsilon	desvio-padrão mínimo da energia na janela
<code>sector_bias = 0.8</code>	beta	prob. de mexer num setor de CD problemático
<code>destination_bias = 0.7</code>	gama	prob. de mandar o setor pra combinação com folga
<code>penalty_coefficient = 1e6</code>	rho	multiplicador das penalidades de inviabilidade

O `100_000` é só `100000` — o `_` é separador de milhar, o Python ignora.

2.4 AnnealingResult (linhas 31-41)

```
@dataclass(frozen=True)
class AnnealingResult:
    assignment: dict[int, int] # setor -> combinacao (a resposta)
    energy: float # E(X) da melhor solucao
    objective: float # z_max - z_min (sem penalidades)
    capacity_penalty: float
    churn_penalty: float
    iterations: int
    stop_reason: str # por que parou
```

O “pacote de saída”. Note que não devolve só `assignment` — devolve a decomposição (`objective`, `capacity_penalty`, `churn_penalty`) porque o documento de negócio pede comparação As-Is vs To-Be com métricas claras.

2.5 solve() — a porta de entrada (linhas 44-76)

```
def solve(sectors, combinations, days, cd_sectors, daily_capacity,
          projected_demand, current_assignment, max_churn,
          valid_combinations=None, params=None, rng=None) -> AnnealingResult:
    params = params or AnnealingParams()
    rng = rng or random.Random()
    problem = _build_problem(sectors, combinations, days, cd_sectors,
                             daily_capacity, projected_demand,
                             current_assignment, max_churn, valid_combinations)
    return _anneal(problem, params, rng)
```

- Os parâmetros são os dados do problema, como listas/dicionários Python normais (rótulos externos: setor “137”, combinação “3”).
 - `current_assignment` mapeia setor → combinação atual: é o As-Is e o ponto de partida da busca.
 - `x or y` retorna `x` se for “verdadeiro”, senão `y`. Então: “se ninguém passou `params`, usa os padrões”. Passar um `rng` com semente fixa (`random.Random(42)`) torna a execução reproduzível — essencial nos testes.
 - Duas fases: **traduzir** os dados para arrays numpy (`_build_problem`), depois **rodar** o algoritmo (`_anneal`).
-

2.6 _Problem — a representação interna (linhas 79-88)

```
@dataclass(frozen=True)
class _Problem:
    demand: np.ndarray          # (n_setores, n_dias, n_combinacoes) q[s,a,d]
    capacity: np.ndarray         # (n_cds, n_dias) Cap[c,a]
    sector_cd: np.ndarray        # (n_setores,) indice do CD de cada setor
    valid_combos: list[np.ndarray] # por CD, os indices de combinacao validos (D_c)
    as_is: np.ndarray            # (n_setores,) indice da combinacao atual de cada setor
    max_churn: float
    sector_labels: list[int]     # pra traduzir indice -> rotulo no final
    combo_labels: list[int]
```

Tudo em **índices de array** (0, 1, 2...) em vez de rótulos, porque numpy só trabalha com posições. O `_` no começo (`_Problem`) é convenção do Python para “isso é interno, não faz parte da API pública”.

Por que agrupar num dataclass em vez de passar 8 variáveis soltas? Porque toda função auxiliar (`_evaluate`, `_neighbor`, `_pick_sector`...) precisaria receber os 8 argumentos. Com o `_Problem`, é só `problem`. E acesso por nome (`problem.demand`) pega tipo na hora, dá autocomplete, e documenta o tipo — coisa que `problem["demand"]` (dicionário) não faz.

2.7 _Evaluation — o resultado de avaliar uma solução (linhas 91-97)

```
@dataclass(frozen=True)
class _Evaluation:
    objective: float          # z_max - z_min
    capacity_penalty: float   # P_cap
    churn_penalty: float      # P_churn
    energy: float             # objective + rho*(P_cap + P_churn)
    load_by_cd: np.ndarray    # (n_cds, n_dias) - guardado pra reaproveitar
```

Ao avaliar uma solução, calculamos `load_by_cd` (carga por CD por dia). A função de vizinhança **precisa** disso (para saber quais CDs estão estourados). Guardar aqui evita recalcular. Sem esse dataclass, `_evaluate` devolveria uma tupla de 5 elementos e todo chamador teria que desempacotar na ordem exata — trocar a ordem sem querer daria bug silencioso.

2.8 _SearchState — o estado da busca (linhas 100-106)

```
@dataclass
class _SearchState:
    current: np.ndarray        # a solucao onde a busca esta agora
    current_eval: _Evaluation
    best: np.ndarray           # a melhor solucao ja vista
    best_eval: _Evaluation
    energy_window: deque[float] = field(default_factory=deque)
```

- Não é frozen porque o `energy_window` (`deque`) é mutável.
- `field(default_factory=deque)`: você **não pode** usar `= deque()` como padrão numa dataclass — seria criado uma vez só e **todas** as instâncias compartilhariam o mesmo deque. O `default_factory` diz “crie um novo a cada instância”.
- **current vs best**: a busca “caminha” (o `current` muda toda hora, às vezes para pior), mas o `best` só é atualizado quando encontra algo melhor que tudo que já viu. No fim, devolvemos o `best`.

2.9 A avaliação da energia (Seção 4.1.3)

daily_load_by_sector (linhas 112-115)

```
def _daily_load_by_sector(problem, assignment):
    sectors = np.arange(assignment.shape[0]) # [0, 1, 2, ..., n-1]
    return problem.demand[sectors, :, assignment]
```

`assignment` é um array tipo `[0, 1, 0, 1, ...]` (a combinação de cada setor). `problem.demand` tem shape `(n_setores, n_dias, n_combinações)`.

`problem.demand[sectors, :, assignment]` é uma **indexação avançada** do numpy: para cada posição `i`, pega `demand[sectors[i], :, assignment[i]]` — ou seja, “toda a linha de dias do setor `i`, na combinação que ele escolheu”. Resultado: shape `(n_setores, n_dias)` — a demanda de cada setor em cada dia, já considerando a combinação atribuída. É o “sumproduct” do MIP, mas direto: como a solução é um vetor de escolhas (não uma matriz de 0/1), é só “pegar a coluna certa”.

load_by_cd (linhas 118-122)

```
def _load_by_cd(problem, load_by_sector):
    totals = np.zeros_like(problem.capacity) # zeros com shape (n_cds, n_dias)
    np.add.at(totals, problem.sector_cd, load_by_sector)
    return totals
```

`np.add.at(totals, indices, valores)` é um “somar acumulando”: para cada setor `i`, faz `totals[sector_cd[i]] += load_by_sector[i]`. Joga a demanda diária de cada setor na linha do CD dele. Resultado: Carga(c,a) do PDF.

Por que `np.add.at` e não `totals[sector_cd] += load_by_sector`? Porque se dois setores estão no mesmo CD, a versão normal “sobrescreveria” a primeira soma. O `.at` força acumular direito.

capacity_penalty (linhas 125-127)

```
def _capacity_penalty(problem, load_by_cd):
    return float(np.maximum(0.0, load_by_cd - problem.capacity).sum())
```

`load_by_cd - problem.capacity` → quanto cada (CD, dia) passou (negativo = tem folga).

`np.maximum(0.0, ...)` → zera as folgas, mantém só os estouros. `.sum()` → total de estouro. É o `P_cap` do PDF: soma de $\max(0, \text{carga} - \text{Cap})$.

churn_penalty (linhas 130-133)

```
def _churn_penalty(problem, assignment):
    changed = int(np.count_nonzero(assignment != problem.as_is))
    return max(0.0, 2.0 * changed - 2.0 * problem.max_churn)
```

`assignment != problem.as_is` → array de True/False (quais setores mudaram). `np.count_nonzero(...)` → número de setores realocados. A fórmula do PDF dá **2·changed** (cada mudança “conta 2”: o setor perde 1 ponto na combinação antiga que abandonou e ganha 1 na nova). Então a penalidade é $\max(0, 2 \cdot \text{changed} - 2M)$: só pune o que passa do teto `M`.

evaluate — junta tudo (linhas 136-145)

```
def _evaluate(problem, assignment, penalty_coefficient):
    load_by_sector = _daily_load_by_sector(problem, assignment)
    daily_totals = load_by_sector.sum(axis=0) # soma sobre setores -> vetor de dias
    load_by_cd = _load_by_cd(problem, load_by_sector)
    objective = float(daily_totals.max() - daily_totals.min()) # z_max - z_min
    capacity_penalty = _capacity_penalty(problem, load_by_cd)
    churn_penalty = _churn_penalty(problem, assignment)
    energy = objective + penalty_coefficient * (capacity_penalty + churn_penalty)
    return _Evaluation(objective, capacity_penalty, churn_penalty, energy, load_by_cd)
```

`daily_totals = load_by_sector.sum(axis=0)` — `axis=0` é o eixo dos setores, então isso soma “todos os setores”, sobrando um valor por dia. `z_max/z_min` são o max/min disso.

`energy = objetivo + rho·(penalidades)` — a função de energia $E(X)$ da Seção 4.1.3. Com $\text{rho} = 1e6$, qualquer inviabilidade domina completamente: uma solução com `P_cap = 1` tem energia $\geq 1.000.000$, então o SA sempre prefere qualquer solução viável.

2.10 A função de vizinhança (Seção 4.1.2)

_violated_cds (linhas 151-153)

```
def _violated_cds(problem, load_by_cd):  
    return np.where((load_by_cd > problem.capacity).any(axis=1))[0]
```

`load_by_cd > problem.capacity` → matriz (`n_cds`, `n_dias`) de True/False. `.any(axis=1)` → para cada CD, “estourou em algum dia?” (`axis=1` é o eixo dos dias). `np.where(...)[0]` → os índices dos CDs que deram True. É o B(X) do PDF.

_pick_sector (linhas 156-163)

```
def _pick_sector(problem, violated_cds, params, rng):  
    overloaded = np.where(np.isin(problem.sector_cd, violated_cds))[0]  
    if overloaded.size and rng.random() < params.sector_bias:  
        return int(rng.choice(overloaded))  
    return rng.randrange(problem.as_is.shape[0])
```

`np.isin(problem.sector_cd, violated_cds)` → para cada setor, “o CD dele está na lista de violados?”.
`np.where(...)[0]` → índices desses setores. É o O(X) do PDF.

`if overloaded.size and rng.random() < params.sector_bias:` — se existe setor problemático **E** o sorteio caiu abaixo de `beta=0.8` → escolhe um deles. Senão → `rng.randrange(n)` sorteia **qualquer** setor. Cobre os dois casos do PDF: 80% intensificação (mexe onde está o problema), 20% diversificação. Se O(X) está vazio, `overloaded.size` é 0 → sempre cai no `randrange`.

_slack_destinations (linhas 166-175)

```
def _slack_destinations(problem, sector, assignment, load_by_cd):  
    cd = int(problem.sector_cd[sector])  
    candidates = problem.valid_combos[cd] # D_c  
    without_sector = load_by_cd[cd] - problem.demand[sector, :, assignment[sector]]  
    projected = without_sector[None, :] + problem.demand[sector][:, candidates].T  
    feasible = (projected <= problem.capacity[cd]).all(axis=1)  
    return candidates[feasible]
```

- `cd` = o CD do setor que vamos mover.
- `candidates` = as combinações válidas para esse CD (`D_c`).
- `without_sector` = carga atual do CD **menos** a contribuição do setor na combinação que ele usa hoje. “Como ficaria o CD se tirássemos esse setor”.
- `projected` = para **cada** combinação candidata, a carga do CD **se colocássemos o setor lá**. `problem.demand[sector]` tem shape (`n_dias`, `n_combinações`); `[:, candidates]` pega só as colunas das candidatas; `.T` transpõe para (`n_candidatas`, `n_dias`); `without_sector[None, :]` vira (`1`, `n_dias`) para o broadcast; a soma dá (`n_candidatas`, `n_dias`): linha `k` = carga do CD em cada dia se o setor fosse para a candidata `k`.
- `feasible` = para cada candidata, “não estoura em nenhum dia?” (`all` sobre o eixo dos dias). `candidates[feasible]` → só as que passam. É o F(X) do PDF.

Nota: a fórmula literal do F(X) no PDF não dependia da combinação `d`, o que não fazia sentido (parecia erro de digitação, como o `D` vs `D_c` do MIP). Aqui foi implementada a intenção óbvia: “destino que mantém o CD viável”.

_pick_destination (linhas 178-191)

```
def _pick_destination(problem, sector, assignment, load_by_cd, params, rng):  
    cd = int(problem.sector_cd[sector])  
    slack = _slack_destinations(problem, sector, assignment, load_by_cd)  
    if slack.size and rng.random() < params.destination_bias:  
        return int(rng.choice(slack))  
    return int(rng.choice(problem.valid_combos[cd]))
```

Mesma estrutura do `_pick_sector`: se existe destino com folga **E** sorteio `< gama=0.7` → escolhe entre os de folga. Senão → escolhe entre **todas** as válidas de `D_c` (incluindo as sobrecarregadas — o “movimento de piora controlada” do PDF). Se F(X) está vazio, `slack.size` é 0 → cai direto no segundo

return (o “relaxa gama para 0” do PDF).

_neighbor (linhas 194-207)

```
def _neighbor(problem, assignment, evaluation, params, rng):
    violated = _violated_cds(problem, evaluation.load_by_cd)
    sector = _pick_sector(problem, violated, params, rng)
    destination = _pick_destination(problem, sector, assignment,
                                    evaluation.load_by_cd, params, rng)

    neighbor = assignment.copy()
    neighbor[sector] = destination
    return neighbor
```

Junta os passos: acha CDs violados → escolhe setor → escolhe destino → **copia** a solução atual e muda **só uma posição**. `assignment.copy()` é essencial — sem isso, mexeríamos na solução atual em vez de criar uma nova.

2.11 O laço principal (Seções 4.1.3 e 4.1.4)

_accepts — Metropolis (linhas 213-217)

```
def _accepts(delta_energy, temperature, rng):
    if delta_energy < 0:
        return True
    return math.exp(-delta_energy / temperature) > rng.random()
```

$\text{delta_energy} < 0 \rightarrow$ o vizinho é melhor $\rightarrow$ aceita sempre. Senão $\rightarrow$ calcula $e^{(-\Delta E/T)}$ (um número entre 0 e 1) e compara com um sorteio uniforme em $[0, 1)$. Se a exponencial for maior, aceita. Quanto maior T (ou menor ΔE), maior a chance.

_stagnated (linhas 220-224)

```
def _stagnated(energy_window, params):
    if len(energy_window) < params.stagnation_window:
        return False
    return float(np.std(energy_window)) < params.stagnation_tolerance
```

Se a janela ainda não encheu (menos de $L=200$ energias), retorna False. Quando cheia, calcula o **desvio-padrão** das 200 últimas energias aceitas. Se estiver quase zero ($< \epsilon$), significa “as soluções estão todas energeticamente iguais, não vou melhorar mais” $\rightarrow$ para.

_accept (linhas 227-233)

```
def _accept(state, assignment, evaluation):
    state.energy_window.append(evaluation.energy)
    best, best_eval = state.best, state.best_eval
    if evaluation.energy < best_eval.energy:
        best, best_eval = assignment.copy(), evaluation
    return _SearchState(assignment, evaluation, best, best_eval, state.energy_window)
```

Chamado **só quando o movimento foi aceito**. Registra a energia na janela. Se a nova energia é a melhor de todas $\rightarrow$ atualiza best (com `.copy()`), senão o best mudaria junto com o current depois). Retorna um **novo** `_SearchState` — o current agora é o vizinho aceito.

_step — uma iteração (linhas 236-249)

```
def _step(problem, state, params, rng, temperature):
    candidate = _neighbor(problem, state.current, state.current_eval, params, rng)
    candidate_eval = _evaluate(problem, candidate, params.penalty_coefficient)
    delta = candidate_eval.energy - state.current_eval.energy
    if not _accepts(delta, temperature, rng):
        return state # rejeitado: nada muda
    return _accept(state, candidate, candidate_eval) # aceito: avanca
```

Gera vizinho $\rightarrow$ avalia $\rightarrow$ calcula $\Delta E \rightarrow$ Metropolis decide. Se rejeitou, devolve o state **inalterado** (a busca fica onde estava). Se aceitou, delega para o `_accept`.

_anneal — o loop (linhas 252-269)

```
def _anneal(problem, params, rng):
    initial = _evaluate(problem, problem.as_is, params.penalty_coefficient)
    state = _SearchState(current=problem.as_is.copy(), current_eval=initial,
                        best=problem.as_is.copy(), best_eval=initial,
                        energy_window=deque(maxlen=params.stagnation_window))
    temperature = params.initial_temperature
    for iteration in range(params.max_iterations):
        if temperature <= params.min_temperature:
            return _build_result(problem, state, iteration, "min_temperature")
        state = _step(problem, state, params, rng, temperature)
        if _stagnated(state.energy_window, params):
            return _build_result(problem, state, iteration + 1, "stagnation")
        temperature *= params.cooling_rate # T <- alpha*T
    return _build_result(problem, state, params.max_iterations, "max_iterations")
```

- Começa **do As-Is** — tanto current quanto best. É isso que garante que o SA nunca devolve algo pior que a situação atual: o As-Is já está guardado como melhor desde o início.
- `deque(maxlen=200)` — quando você adiciona o 201º elemento, ele descarta automaticamente o mais antigo. É a “janela deslizante”.

- O for vai até `MaxIter`, mas há duas saídas antecipadas dentro: temperatura mínima e estagnação.
- `temperature *= params.cooling_rate` no fim de cada volta — o resfriamento geométrico $T(k+1) = \alpha \cdot T(k)$.

`_build_result` (linhas 272-287)

```
def _build_result(problem, state, iterations, stop_reason):
    assignment = {problem.sector_labels[i]: problem.combo_labels[int(d)]
                  for i, d in enumerate(state.best)}
    ev = state.best_eval
    return AnnealingResult(assignment=assignment, energy=ev.energy,
                           objective=ev.objective, capacity_penalty=ev.capacity_penalty,
                           churn_penalty=ev.churn_penalty, iterations=iterations,
                           stop_reason=stop_reason)
```

`state.best` é um array de índices tipo `[0, 1, 0, ...]`. O dict comprehension **traduz de volta**: posição `i` → rótulo do setor (`sector_labels[i]`), valor `d` → rótulo da combinação (`combo_labels[d]`). Resultado: `{137: 3, 138: 1, ...}` com os rótulos de verdade. Empacota tudo no `AnnealingResult`.

2.12 A construção do problema (linhas 293-385)

_build_problem (linhas 293-324)

```
sector_labels = list(sectors)      # [137, 138, 139, ...]
combo_labels = list(combinations)
cd_labels = list(cd_sectors)      # as chaves do dict = os CDs
sector_ix = {s: i for i, s in enumerate(sector_labels)}  # {137: 0, 138: 1, ...}
combo_ix = {d: i for i, d in enumerate(combo_labels)}
# ... e monta o _Problem chamando um helper por array
```

Cria os **mapas rótulo → índice**. `enumerate` dá pares (posição, valor). `list(cd_sectors)` num dict retorna as **chaves** (os CDs). Depois monta o `_Problem` chamando um helper por array — cada um uma função pequena e de propósito único (convenção do repo).

_demand_array (linhas 327-336)

```
demand = np.zeros((len(sector_ix), len(day_ix), len(combo_ix)))
for (s, a, d), qty in projected_demand.items():
    demand[sector_ix[s], day_ix[a], combo_ix[d]] = qty
return demand
```

Cria um array 3D de zeros e preenche. Para cada entrada (setor, dia, combinação) → quantidade do dicionário, traduz os rótulos para índices e coloca o valor na posição certa. Chaves que não aparecem no dicionário ficam **0** (demanda zero).

_capacity_array (linhas 339-347)

Igual, mas 2D (`n_cds`, `n_dias`), preenchido a partir do dict (CD, dia) → capacidade.

_sector_cd_array (linhas 350-359)

```
sector_cd = np.zeros(len(sector_ix), dtype=int)
for c, members in cd_sectors.items():
    for s in members:
        sector_cd[sector_ix[s]] = cd_ix[c]
return sector_cd
```

`cd_sectors` é {CD: [lista de setores]}. O loop duplo percorre cada CD e cada setor dele, gravando “o setor na posição `sector_ix[s]` pertence ao CD `cd_ix[c]`”. Resultado: vetor onde `sector_cd[i]` = índice do CD do setor `i`.

_as_is_array (linhas 362-370)

```
as_is = np.zeros(len(sector_ix), dtype=int)
for s, d in current_assignment.items():
    as_is[sector_ix[s]] = combo_ix[d]
return as_is
```

`current_assignment` é {setor: combinacao_atual}. Traduz para vetor de índices: `as_is[i]` = índice da combinação atual do setor `i`. É o X(As-Is) do PDF, na forma compacta.

_valid_combos_by_cd (linhas 373-385)

```
combos = [np.arange(n_combos) for _ in cd_labels]  # padrao: todo CD aceita tudo
if valid_combinations is None:
    return combos
for c, allowed in valid_combinations.items():
    combos[cd_ix[c]] = np.array([combo_ix[d] for d in allowed], dtype=int)
return combos
```

Começa assumindo que **todo CD aceita todas as combinações** (`np.arange(n_combos) = [0, 1, ..., n-1]`). Se o usuário passou `valid_combinations` (o `D_c` de verdade), sobrescreve para os CDs que ele especificou. CDs não mencionados mantêm “aceita tudo”.

Parte 3 — Trace passo a passo (com números)

Instância pequena: 2 setores, 2 combinações, 2 dias, 1 CD. Combinação 1 joga toda a demanda no dia 1; combinação 2 joga no dia 2. As-Is: os dois setores na combinação 1.

Passo 0 — `_build_problem` monta os arrays

```
demand[setor, dia, combinacao] (shape 2x2x2):
  setor 0: dia 0 -> [combo0=10, combo1=0]      dia 1 -> [combo0=0, combo1=10]
  setor 1: dia 0 -> [combo0=10, combo1=0]      dia 1 -> [combo0=0, combo1=10]
capacity      = [[100, 100]]
sector_cd     = [0, 0]
valid_combos  = [ [0, 1] ]
as_is        = [0, 0]
```

Passo 1 — avaliação do ponto de partida: `_evaluate(X = [0, 0])`

```
_daily_load_by_sector: setor 0 usa combo 0 -> [10, 0]; setor 1 usa combo 0 -> [10, 0]
daily_totals = [20, 0]
objective    = 20 - 0 = 20                                <- z_max - z_min
load_by_cd   = [[20, 0]]
capacity_penalty = max(0, [20,0] - [100,100]) somado = 0
churn_penalty  = 2*(0 mudaram) - 2*2 = -4 -> max(0, -4) = 0
energy = 20 + 1e6*(0 + 0) = 20
```

Estado inicial: current = [0,0] energia 20, best = [0,0] energia 20, temperature = 1000.

Passo 2 — Iteração 0: gera vizinho (`_neighbor`)

- **_violated_cds:** [[20,0]] > [[100,100]] → tudo False → nenhum CD em violação → B(X) = [].
- **_pick_sector:** como O(X) está vazio, pula o viés beta e sorteia qualquer setor → digamos **setor 1**.
- **_pick_destination para o setor 1:** without_sector = [20,0] - [10,0] = [10,0]. Projeta: destino combo 0 → [20,0] (OK), destino combo 1 → [10,10] (OK). F(X) = [0, 1]. Sorteio < gama=0.7 → escolhe combo 1.
- **monta o vizinho:** neighbor = [0,0].copy(), neighbor[1] = 1 → **X' = [0, 1]**.

Passo 3 — Iteração 0: avalia o vizinho `_evaluate(X' = [0, 1])`

```
setor 0 usa combo 0 -> [10, 0]; setor 1 usa combo 1 -> [0, 10]
daily_totals = [10, 10]
objective    = 10 - 10 = 0                                <- nivelou!
capacity_penalty = 0
churn_penalty  = 2*(1 mudou) - 4 = -2 -> max(0, -2) = 0
energy = 0
```

Passo 4 — Iteração 0: aceita? (`_accepts`)

```
delta = 0 - 20 = -20
delta < 0 -> aceita direto (e uma melhora)
```

`_accept`: adiciona 0 na janela; 0 < 20 → **best = [0, 1]**. current vira [0, 1].

Passo 5 — fim da Iteração 0

```
_stagnated? janela tem 1 valor, precisa de 200 -> nao
temperature = 1000 * 0.99 = 990
```

O que acontece daí para frente

O current fica orbitando as soluções de energia 0 ([0,1] e [1,0], que também dá amplitude 0). De vez em quando, com temperatura alta, aceita uma piora temporária para [0,0] ou [1,1] (energia 20) — mas volta nas iterações seguintes. Conforme T cai, para de aceitar as piores.

Para quando: a janela de 200 energias aceitas fica quase toda em 0 → desvio-padrão < epsilon → stop_reason = "stagnation". (Ou, se demorar, T chega em 0.01 na iteração ~1146 → "min_temperature".)

Parte 4 — Mapa: formulação (PDF) ↔ código

Seção do PDF	Onde está no código
3.1 Conjuntos (S, D, A, C, S_c, D_c)	_build_problem + arrays; valid_combos = D_c
3.2 Parâmetros (q, Cap, x-AsIs, M)	_demand_array, _capacity_array, _as_is_array, _Problem.max_churn
3.3 Variáveis (x, z_max, z_min)	assignment (vetor de índices); z_max/z_min calculados em _evaluate
3.4 Objetivo (min z_max – z_min)	_evaluate: objective = daily_totals.max() - min()
3.5.1 Atribuição única	estrutural: cada posição do vetor tem exatamente um valor
3.5.2 / 3.5.3 z_max / z_min	_evaluate: max/min dos totais diários
3.5.4 Capacidade por CD	_capacity_penalty (virou penalidade, não restrição dura)
3.5.5 Controle de churn	_churn_penalty (idem)
4.1.1 Espaço de busca (vetor X)	assignment: np.ndarray de índices de combinação
4.1.2 Função de vizinhança	_neighbor, _violated_cds, _pick_sector, _pick_destination
4.1.2 B(X), O(X), F(X)	_violated_cds, isin em _pick_sector, _slack_destinations
4.1.3 Energia E(X) e penalidades	_evaluate, _capacity_penalty, _churn_penalty
4.1.3 Metropolis	_accepts
4.1.4 Resfriamento geométrico	_anneal: temperature *= cooling_rate
4.1.4 Critérios de parada (3)	_anneal (T_min, MaxIter) + _stagnated
4.1.5 Interface / caixa preta	_evaluate isolada do laço; SA só vê o escalar energy

Parte 5 — Notas de design

- **numpy em vez de Python puro:** 100k iterações × somatórios sobre 800×30 seria minutos em Python puro; com numpy vetorizado, cada avaliação é ~microsegundos (smoke test: 1146 iterações em 0.11s).
- **AnnealingParams:** todos os “placeholders a calibrar” do PDF são campos com valor padrão, fáceis de mudar sem tocar no algoritmo.
- **Interpretação do F(X):** a fórmula literal do PDF tinha uma imprecisão (não dependia de d); foi implementada a intenção — “destino que mantém o CD viável”.
- **O resfriamento para em ~1146 iterações** com os parâmetros do PDF (T0=1000, alfa=0.99 por iteração, T_min=0.01): T cai rápido demais para chegar perto de MaxIter. Não é bug, é a spec — e o PDF marca esses valores como placeholders. Para 800 setores, ~1146 iterações é pouco (~1.4 movimentos por setor); provavelmente vão ter que ajustar alfa ou fazer várias iterações por nível de temperatura.
- **@dataclass:** usado nas 4 classes de dados só para não escrever o construtor e o __repr__/_eq__ na mão. O resultado é uma classe normal do Python.
- **Restrições viraram penalidades:** no MIP, capacidade e churn são restrições duras (o solver nunca as viola). No SA, viram termos de penalidade na energia, com peso rho grande — o algoritmo *pode* passar por soluções inviáveis durante a busca, mas elas têm energia altíssima, então ele foge delas.